

Using Netkit & Wireshark**This Week's Objectives**

1. Use Netkit, tcpdump and Wireshark to capture and analyse network traffic packets.
-

Expected Outcomes

After the laboratory session, students should be able to:

- Open a Linux terminal and execute multiple Linux commands.
 - Execute a simple Netkit lab with two machines to inter-communicate.
 - Capture network traffic (packets) into a pcap file which is accessible in the host system.
 - Review the contents a captured pcap file using Wireshark.
-

Workplan

These lab worksheets assume that you will make accurate notes of everything that you do for later consideration and revision. You should use your lab diary to note down items such as commands used, the outcome of each command, and answers to questions posed in these instructions.

All lab machines have CherryTree Notes software installed, which facilitates organised note-taking. Alternatively, you could buy a hardback A4 laboratory notebook.

NB this is a complex lab – please read each instruction carefully – if you miss out any steps, then don't expect to achieve the correct results! You will need to have completed and thoroughly understood all the steps before the next lab session.

Remember that there is a lab work wiki available to you on BB. Please use this to share and communicate with your colleagues.

Make sure you have completed all the steps in this worksheet before the next lab session.

NB The lab workshops require you to use a tool called Netkit. However, the point of using this tool is to capture packets that will allow you to analyse and understand the formats and information carried by frames for each of the different protocols we will be learning. Do not over-emphasise Netkit to the detriment of the actual objective of the module, which is to understand how the Internet is used to share and transport information.

Lab Worksheet 10
Netkit & Wireshark**Linux**

This module uses Linux extensively. It is strongly suggested that you build a glossary of the commands that you use.

1. If part of an instruction needs you to enter a specific value, the place holder will be shown in italics like this (when entering a value, you omit the << and >>):

```
ping c1 <<target machine's ip address>>
```

Netkit

2. Netkit is a very useful wrapper around Jeff Dike's User Mode Linux. It makes the configuration of virtual network devices fairly straightforward (once you get the hang of it - it can take a bit of getting used to initially). In effect, Netkit allows an unprivileged user (for example you when sitting at a University machine) to do all sorts of Network configuration that would normally require root (Unix jargon for what Windows users would describe as "administrator") privilege.
3. The Netkit version installed in the lab machines is Netkit-JH (<https://netkit-jh.github.io/>), an updated version of the original netkit
4. Netkit can be used in two modes:
 - one machine terminal at a time using v*** instructions
 - several machine terminals (xterms) together, configured as a network, using l*** instructions
5. You will use the single machine v*** instructions in this first lab.
6. Spend a few minutes familiarising yourself with the Netkit-jh man pages [here](#).

Wireshark

Health Warning: You are about to use a very powerful tool with restraint. If you capture the traffic of users without their informed consent, then you probably contravene the Computer Misuse Act.

7. Wireshark is a graphical network traffic analyser. In the lab sessions throughout this module, you will use it to analyse traffic that has been captured from your Netkit virtual machine networks. Wireshark can also be used to capture network traffic. We will not be doing this.
8. In order to present you with a graphical representation of data traversing a network, Wireshark "knows" how to interpret the stream of ones and zeros in many different protocols (Ethernet, IP, TCP, DNS, HTTP etc). The graphical interface then allows you to filter what you want to look at (only DNS traffic for example, or perhaps traffic destined for TCP port 80 on the machine with IP address 146.227.150.225).

Lab Worksheet 10
Netkit & Wireshark**Activities**

Reminder

- You are prepared to make accurate notes of what you do
- You will complete unfinished activities before the next timetabled lab session.
- You will resolve what you do not understand by conducting your own careful (and ethically sound) experimentation and / or further reading.

Horizon vs the Virtual Machine

9. You could complete the lab work *either* in a Horizon remote access window, *or* in a virtual machine (or in both ...).
 - The Horizon interface is available at <https://yourdmulabs.dmu.ac.uk>. Once you have logged in, select **CEM – Linux (Ubuntu)** to continue.
 - All your work will be stored on your H drive account
 - Instructions for downloading the VM file are available from the LZ content area for week 1.
 - All your work will be stored within the VM. I recommend that you backup your work regularly to your OneDrive storage space.

Getting Started with Linux

10. You will be using Ubuntu 2022.04 (LTS) to complete the lab work for this module. When you are ready, take a few minutes to remind yourself of the layout of the Ubuntu desktop
11. Open a terminal and create a directory for this lab's work and move into it.

```
cd ~  
mkdir -p ctec1704/lab_10  
cd ctec1704/lab_10
```

NB the system prompt should now look something like:

```
p99999999@machine_name:~/ctec1704/lab_10$
```

Note:

- your account name
- the address of the machine
- the '~' character to signify the home directory of your account
- the forward '/' to separate directories
- the '\$' character

12. Display the fully qualified path (FQP) again:

```
p99999999@machine_name:~/ctec1704/lab_10$ pwd
```

To understand the *pwd* command, use the *man* command (NB system prompt shorteed for brevity):

```
.../lab_10$ man pwd
```

The *man(ual)* command is a reference tool to the Linux command set. The above command will open a page of information about the *pwd* command. To navigate, use the enter key to move forward one line, the space bar to jump forward a page and, at any time 'q' will quit the tool.

Getting started with Netkit

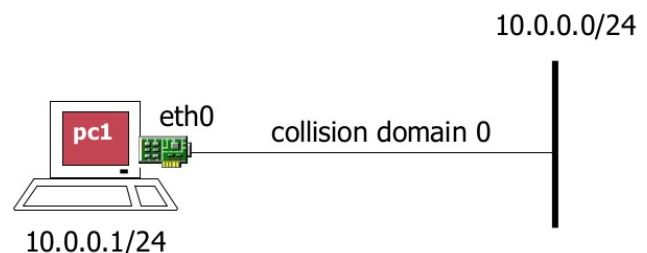
Netkit is a project for creating virtual network devices (routers, switches, servers, etc) which can be connected to form a full network on a single host machine. This can be useful for running network related experiments or for education. Netkit machines are User-mode Linux instances which are lightweight and do not require root access to run.

Netkit is pre-installed on the lab machines. However, there is no icon to access an application window.

To use Netkit simply type the appropriate Netkit command at the system prompt.

Create a single netkit virtual machine

13. The simplest network host device can be visualised as:



14. In the host system terminal window enter the following:

```
.../lab_10$ vstart pc1 --eth0=A
```

This reads as:

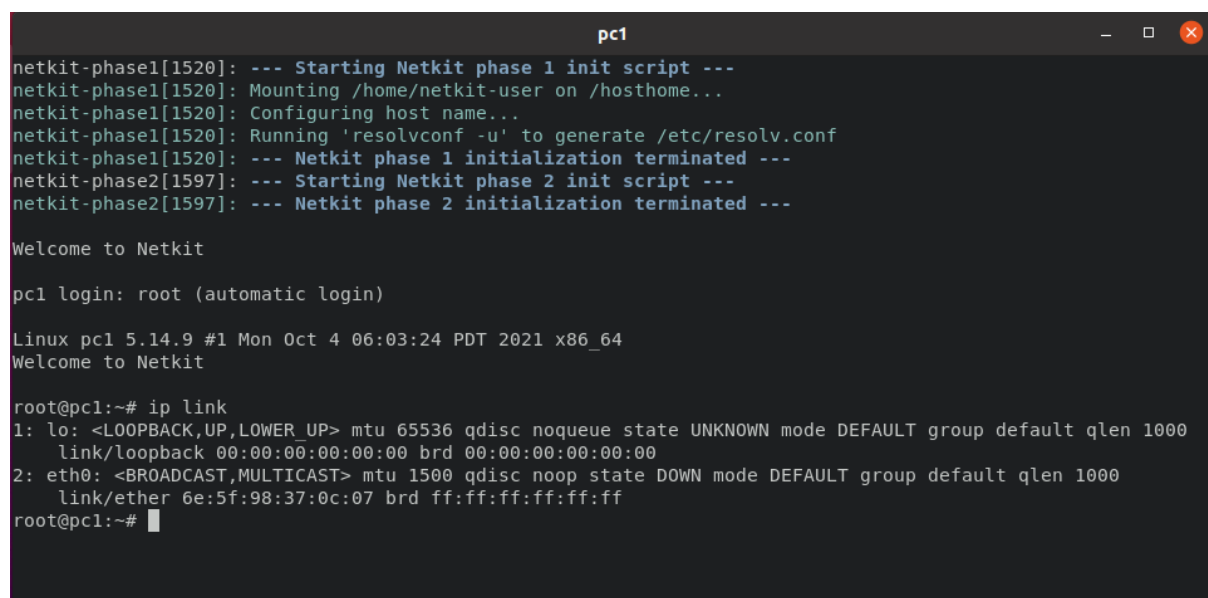
- *vstart*: start a vm
- whose name is *pc1*
- that has a network interface card (NIC) called "*eth0*" on the collision domain called "A"

Lab Worksheet 10
Netkit & Wireshark

NB a virtual filesystem for pc1 is automatically created; it will be stored inside a file called *pc1.disk* on the host machine.

15. When the bootstrap (ie starting up) processes of pc1 are complete, you will see a netkit terminal window with the title of pc1 as per the screenshot below.
NB Note that you are logged in as the root administrator of this terminal.
16. In the netkit terminal **pc1** enter *ip link* to display information about the current network interfaces:

```
root@pc1:~# ip link
```



```

pc1
netkit-phase1[1520]: --- Starting Netkit phase 1 init script ---
netkit-phase1[1520]: Mounting /home/netkit-user on /hoshome...
netkit-phase1[1520]: Configuring host name...
netkit-phase1[1520]: Running 'resolvconf -u' to generate /etc/resolv.conf
netkit-phase1[1520]: --- Netkit phase 1 initialization terminated ---
netkit-phase2[1597]: --- Starting Netkit phase 2 init script ---
netkit-phase2[1597]: --- Netkit phase 2 initialization terminated ---

Welcome to Netkit

pc1 login: root (automatic login)

Linux pc1 5.14.9 #1 Mon Oct 4 06:03:24 PDT 2021 x86_64
Welcome to Netkit

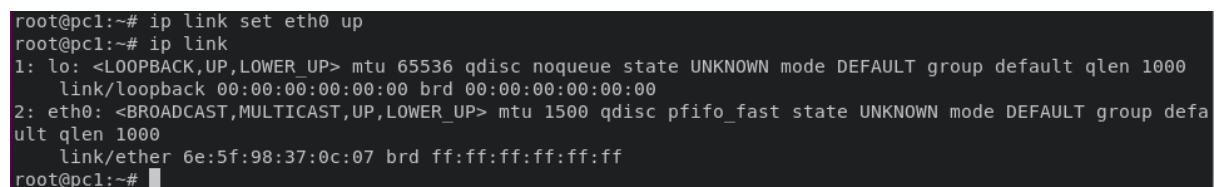
root@pc1:~# ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN mode DEFAULT group default qlen 1000
    link/ether 6e:5f:98:37:0c:07 brd ff:ff:ff:ff:ff:ff
root@pc1:~#

```

17. Note the information about *eth0*. It has a default MAC address (the link/ether setting), and its state is currently DOWN.
18. In the netkit terminal **pc1**, enter:

```
root@pc1:~# ip link set eth0 up
root@pc1:~# ip link
```

you will notice the the state of *eth0* is now UP:



```

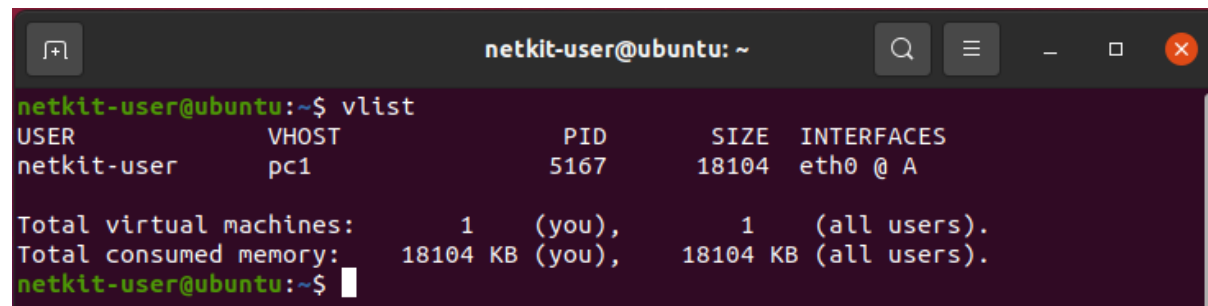
root@pc1:~# ip link set eth0 up
root@pc1:~# ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UNKNOWN mode DEFAULT group default qlen 1000
    link/ether 6e:5f:98:37:0c:07 brd ff:ff:ff:ff:ff:ff
root@pc1:~#

```

Lab Worksheet 10
Netkit & Wireshark

19. Back in the host machine terminal, display the list of currently active netkit virtual machines:

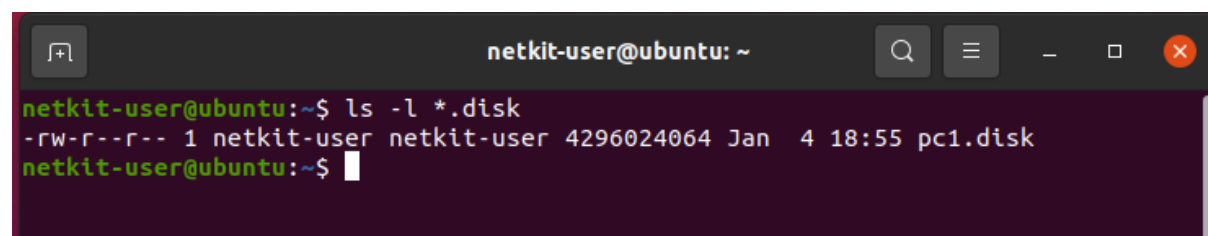
```
.../lab_10$ vlist
```



```
netkit-user@ubuntu: ~  
netkit-user@ubuntu:~$ vlist  
USER          VHOST          PID          SIZE          INTERFACES  
netkit-user    pc1            5167         18104         eth0 @ A  
  
Total virtual machines:      1      (you),      1      (all users).  
Total consumed memory:    18104 KB (you),    18104 KB (all users).  
netkit-user@ubuntu:~$
```

20. Now list the currently active file systems:

```
.../lab_10$ ls -l *.disk
```



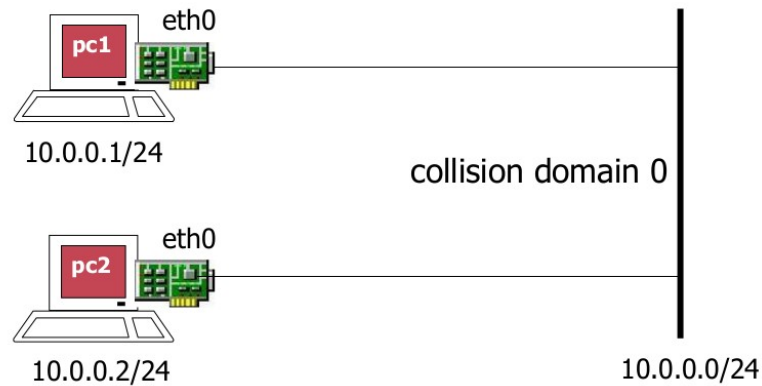
```
netkit-user@ubuntu:~$ ls -l *.disk  
-rw-r--r-- 1 netkit-user netkit-user 4296024064 Jan  4 18:55 pc1.disk  
netkit-user@ubuntu:~$
```

21. To close the netkit terminal, use the **vhalt** command (see <https://netkit-jh.github.io/docs/man/netkit-manpage-vhalt> for information about the switch used). Use the ls command again to confirm that the disk file has been removed.

```
.../lab_10$ vhalt pc1 -r  
.../lab_10$ ls -l *.disk
```

Creating multiple netkit virtual machines

Netkit can be used to simulate different sizes of networks, from just two machines to many dozens – just how many can be created is largely dependent upon the memory capacity of the hosting machine. In this next exercise, you are going to build a small network with just two hosts:



22. In the host system terminal, start two netkit vms:

```
.../lab_10$ vstart pc1 --eth0=A
.../lab_10$ vstart pc2 --eth0=A
```

The two vms are connected to the same collision domain. Therefore, they will be able to message each other, once their respective network interfaces have been configured with an [IP address](#) and a [netmask](#).

23. In the netkit terminal *pc1*, enter:

```
root@pc1:~# ip addr add 10.0.0.1/24 dev eth0
root@pc1:~# ip addr show dev eth0
```

24. The second command displays the IP address and netmask for *pc1*
NB the IP address is **10.0.0.1** and the netmask is **/24**

```
pc1
root@pc1:~# ip addr add 10.0.0.1/24 dev eth0
root@pc1:~# ip addr show dev eth0
2: eth0: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether 6e:5f:98:37:0c:07 brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.1/24 scope global eth0
        valid_lft forever preferred_lft forever
root@pc1:~#
```

25. Repeat the process for *pc2*, but change the IP address:

```
root@pc2:~# ip addr add 10.0.0.2/24 dev eth0
root@pc2:~# ip addr show dev eth0
```

26. In both of the netkit vms, change the state of the network interface, as before:

```
root@pc1:~# ip link set eth0 up
root@pc1:~# ip addr show dev eth0
```

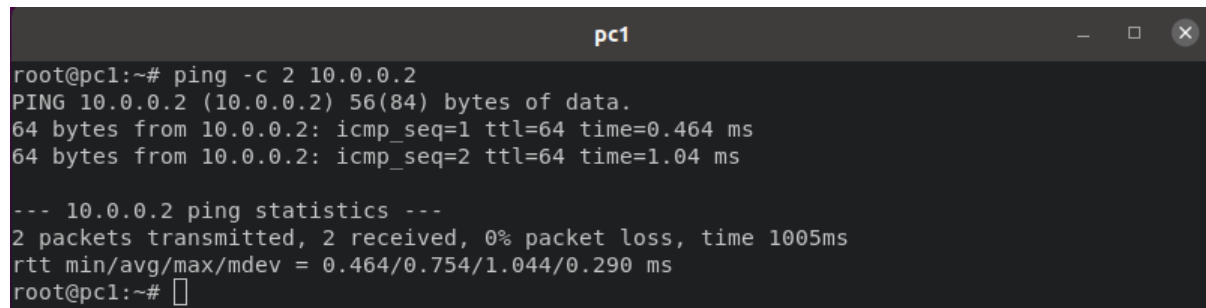
Sending/receiving messages

Use the *ping* command to communicate between the netkit xterms (*man ping* to read about the command). The ping command is used

27. In *pc1*, send exactly 2 pings to *pc2*:

```
root@pc1:~# ping -c 2 10.0.0.2
```

And the output:



```
pc1
root@pc1:~# ping -c 2 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.464 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=1.04 ms

--- 10.0.0.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1005ms
rtt min/avg/max/mdev = 0.464/0.754/1.044/0.290 ms
root@pc1:~#
```

Using tcpdump to capture messages

The tcpdump network sniffer tool is used to sniff ethernet frames as they arrive/depart a network interface. Sniffed message frames can be stored for later analysis.

28. Start tcpdump in *pc2* (as usual, *man tcpdump* to understand the switches used)

```
root@pc2:~# tcpdump -v -i eth0
```

29. Resend the two pings from *pc1* (Ctrl C in *pc2* will stop the capture):

Lab Worksheet 10 Netkit & Wireshark

Output on pc2:

```

root@pc2:~# tcpdump -v -i eth0
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
20:03:47.660318 IP (tos 0x0, ttl 64, id 44365, offset 0, flags [DF], proto ICMP (1), length 84)
    10.0.0.1 > 10.0.0.2: ICMP echo request, id 27999, seq 1, length 64
20:03:47.660339 IP (tos 0x0, ttl 64, id 5447, offset 0, flags [none], proto ICMP (1), length 84)
    10.0.0.2 > 10.0.0.1: ICMP echo reply, id 27999, seq 1, length 64
20:03:48.729212 IP (tos 0x0, ttl 64, id 44445, offset 0, flags [DF], proto ICMP (1), length 84)
    10.0.0.1 > 10.0.0.2: ICMP echo request, id 27999, seq 2, length 64
20:03:48.729262 IP (tos 0x0, ttl 64, id 5451, offset 0, flags [none], proto ICMP (1), length 84)
    10.0.0.2 > 10.0.0.1: ICMP echo reply, id 27999, seq 2, length 64
^C
4 packets captured
4 packets received by filter
0 packets dropped by kernel
root@pc2:~#

```

30. tcpdump will not only capture the messages, but will also store them in a *pcap* file for later analysis. Create a directory in your host system for this. In the host terminal:

```
.../lab_10$ mkdir -p ~/ctec1704/captures
```

31. In *pc2*, start tcpdump again, with the following switches:

```
root@pc2:~# tcpdump -v -i eth0 -s0 -w /hosthome/ctec1704/captures/ping1.pcap
```

```

root@pc2:~# tcpdump -v -i eth0 -s0 -w /hosthome/ctec1907/captures/ping1.pcap
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C8 packets captured
8 packets received by filter
0 packets dropped by kernel
root@pc2:~#

```

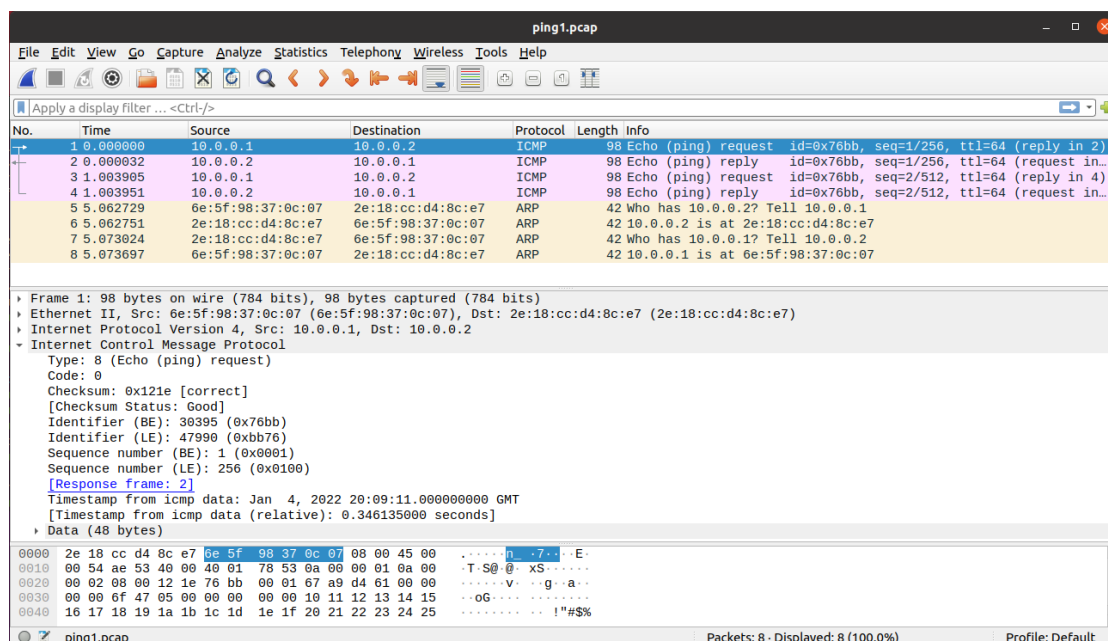
32. Confirm that a *pcap* file has been created in the *captures* directory:

```
.../lab_10$ ls -l ~/ctec1704/captures
```

Wireshark

33. The next step is to analyse the captured messages with the Wireshark application (it used to be known as *ethereal*), pre-installed on the Ubuntu desktop.
34. Execute Wireshark (how?), and open the pcap file you saved:

Lab Worksheet 10 Netkit & Wireshark



35. Wireshark is an extremely powerful and versatile tool used to analyse network traffic. In the screenshot you can see details of the 8 messages captured, the two types of protocol messages used (ICMP and ARP), and the content of the first message.
36. In the middle pane, you can see details of the Internet Protocol layers. Expand the Internet Control Message Protocol details (how?) and compare the information shown for the ping request and response packets.
37. Use a reference web-site (eg wikipedia, tomshardware, etc) to investigate how Ping messages are carried by the ICMP protocol. What other application messages are also carried by the ICMP protocol?
38. Spend some time experimenting with what you can see. There are many tutorials on-line to learn how to use this tool.

Don't forget to take copious notes!!!

Follow-up consolidation and extension

39. Carry out the two xterms lab again but this time give the machines different names and different IP addresses. Note down carefully what you see that is different.
40. Add two more xterms. Give them relevant IP addresses and make sure that each xterm can connect to each of the others.
41. What happens if two xterms have the same IP address? Change the IP address of one xterm to be the same as one of the others. Can the xterms still message each other? Experiment in a systematic way to see if you are correct. Record your findings for later.
42. Draw and thoroughly annotate a diagram of the network you created. Compare this

Lab Worksheet 10
Netkit & Wireshark

with diagrams drawn by your course colleagues.

43. Use the relevant *man* page to look up extra switches (eg *-p*) for the *mkdir* command, and experiment.

NB The *rmdir* command is used to remove unwanted directories.

44. Research the filters that are available within Wireshark.